LAPORAN PRAKTIKUM STRUKTUR DATA

"STACK"

Disusun Oleh:

Aryahiyahul Fikra

2511532026

Dosen Pengampu:

Dr. Wahyudi S.T, M.T

Asisten Pembimbing:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Daftar Isi

Daftar Isi 2
BAB I 3
PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan Praktikum 3
BAB II 4
PEMBAHASAN 4
2.1 Implementasi Stack Menggunakan Array 4
2.2 Driver Stack 5
2.3 Stack Menggunakan ArrayList (SiswaStack) 6
2.4 Mencari Nilai Maksimum dalam Stack 7
2.5 Evaluasi Ekspresi Postfix 8
BAB III 10
PENUTUP 10
3.1 Kesimpulan 10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara efisien. Salah satu struktur data yang sering digunakan adalah **stack (tumpukan)**, yang bekerja dengan prinsip **LIFO (Last In First Out)**, yaitu elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Dalam praktikum ini, dilakukan implementasi stack menggunakan berbagai pendekatan, seperti:

- Stack berbasis array
- Stack menggunakan ArrayList
- Operasi dasar stack (push, pop, peek)
- Pencarian nilai maksimum dalam stack
- Evaluasi ekspresi postfix menggunakan stack

Melalui praktikum ini diharapkan mahasiswa dapat memahami konsep stack dan penerapannya dalam berbagai kasus.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar stack
2. Mengimplementasikan stack dalam bahasa Java
3. Menggunakan operasi push, pop, dan peek
4. Mengaplikasikan stack dalam kasus nyata seperti postfix
5. Menganalisis hasil dari program yang dibuat

BAB II
PEMBAHASAN

2.1 Implementasi Stack Menggunakan Array

```java
package pekan3_2511532026;

public class StackArray_2511532026 {
    static final int MAX = 1000;
    int top;
    int a[] = new int [MAX];
    boolean isEmpty() {
        return (top<0);

    }
    StackArray_2511532026() {
    top = -1;
}
    boolean push_2511532026 (int x) {
        if (top >= (MAX -1)) {
            System.out.println("Stack overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println(x +" dimasukan dalam stack");
            return true;
        }
    }
    int pop_2511532026() {
        if (top<0) {
            System.out.println("sistem underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
    int peek_2511532026() {
        if (top<0) {
            System.out.println("stack underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;

        }
```

```java
    }
    void print_2511532026() {

        for (int i = top;i>-1;i--) {
        System.out.println(" "+a[i]);
        }
    }
}
```

Pada class StackArray_2511532026, stack dibuat menggunakan array dengan kapasitas maksimum 1000.

Operasi yang digunakan:

- push() → menambahkan elemen ke stack
- pop() → menghapus elemen teratas
- peek() → melihat elemen teratas
- isEmpty() → mengecek apakah stack kosong

Stack menggunakan variabel top untuk menunjukkan posisi elemen teratas.

Jika stack penuh → terjadi overflow

Jika stack kosong → terjadi underflow

2.2 Driver Stack

```java
package pekan3_2511532026;

public class StackArrayDriver_2511532026 {

    public static void main(String[] args) {
        StackArray_2511532026 s = new StackArray_2511532026();
        s.push_2511532026(10);
        s.push_2511532026(20);
        s.push_2511532026(30);
        System.out.println(s.pop_2511532026()+ " dikeluarkan dari stack");
        System.out.println("elemen teratas  adalah "+ s.peek_2511532026());
        System.out.println("element pada stack");
        s.print_2511532026();

    }

}
```

Pada class StackArrayDriver_2511532026, dilakukan pengujian stack:

- Menambahkan elemen: 10, 20, 30
- Menghapus elemen teratas
- Menampilkan isi stack

Driver ini berfungsi untuk melihat apakah fungsi stack berjalan dengan benar.

2.3 Stack Menggunakan ArrayList (SiswaStack)

```java
package pekan3_2511532026;
import java.util.ArrayList;

class Siswa {
    String nama;
    int nim;

    public Siswa(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "NIM: " + nim + ", Nama: " + nama;
    }
}

public class SiswaStack_2511532026 {
    private ArrayList<Siswa> stack;


    public SiswaStack_2511532026() {
        stack = new ArrayList<>();
    }

    public void push(Siswa mhs) {
        stack.add(mhs);
    }

    public Siswa pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

```

```java
    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }

    public static void main(String[] args) {
        SiswaStack_2511532026 studentStack = new SiswaStack_2511532026();

        Siswa mhs1 = new Siswa("Ary", 1);
        Siswa mhs2 = new Siswa("ataking", 2);
        Siswa mhs3 = new Siswa("oerel", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa();

        System.out.println("Siswa teratas " + studentStack.peek());
        System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack.pop());
        System.out.println("daftar siswa setelah di pop :");
        studentStack.tampilkanSiswa();
    }
}
```

Pada class SiswaStack_2511532026, stack dibuat menggunakan **ArrayList** dan menyimpan objek Siswa.

Class Siswa:

Memiliki atribut:

- nama
- nim

Operasi:

- push → menambahkan siswa
- pop → menghapus siswa terakhir
- peek → melihat siswa teratas
- tampilkanSiswa → menampilkan isi stack

Keunggulan:

- Lebih fleksibel dibanding array
- Bisa menyimpan objek (tidak hanya integer)

2.4 Mencari Nilai Maksimum dalam Stack

```java
package pekan3_2511532026;
import java.util.*;
public class NilaiMaksimum_2511532026 {

    public static int max(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();

        int maxValue = s.pop();
        backup.push(maxValue);

        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }

        // Mengembalikan isi stack seperti semula
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }

        return maxValue;
    }

    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();

        s.push(70);
        s.push(12);
        s.push(20);
        s.push(26*19);
        s.push(19);
        s.push(26);


        System.out.println("Isi stack " + s);
        System.out.println("Stack Teratas " + s.peek());
        System.out.println("Nilai maksimum " + max(s));
    }
}
```

Pada class NilaiMaksimum_2511532026, dibuat fungsi untuk mencari nilai terbesar dalam stack.

Cara kerja:

1. Ambil elemen pertama sebagai nilai maksimum awal
2. Bandingkan dengan elemen lain
3. Gunakan stack bantu (backup) agar data tidak hilang
4. Kembalikan isi stack seperti semula

Konsep penting:

- Stack tidak boleh berubah setelah proses
- Menggunakan stack tambahan sebagai Solusi

2.5 Evaluasi Ekspresi Postfix

```java
package pekan3_2511532026;
import java.util.*;

public class StackPostFix_2511532026 {

    public static int PostFixEvaluate_2511532026(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner(expression);

        while (input.hasNext()) {
            if (input.hasNextInt()) {
                s.push(input.nextInt());
            } else {
                String operator = input.next();

                int operand2 = s.pop();
                int operand1 = s.pop();

                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
                }
            }
        }

        input.close();
        return s.pop();
    }

    public static void main(String[] args) {
        System.out.println("hasil postfix = " +
            PostFixEvaluate_2511532026("5 2 4 * + 7 -"));
    }
}
```

Pada class StackPostFix_2511532026, digunakan stack untuk menghitung ekspresi postfix.

Contoh:

5 2 4 * + 7 -

Langkah:

1. Jika angka → push ke stack

2. Jika operator:
    - Ambil 2 angka dari stack
    - Lakukan operasi
    - Simpan hasil ke stack

Hasil akhir:

Nilai terakhir dalam stack adalah hasil perhitungan

Kelebihan postfix:

- Tidak perlu tanda kurung
- Mudah diproses dengan stack

BAB III
PENUTUP

3.1 Kesimpulan

Kesimpulan dari praktikum ini adalah bahwa struktur data stack merupakan metode penyimpanan data yang bekerja dengan prinsip LIFO (Last In First Out), di mana elemen terakhir yang dimasukkan akan menjadi yang pertama dikeluarkan. Melalui berbagai implementasi yang telah dilakukan, seperti penggunaan array dan ArrayList, dapat dipahami bahwa stack memiliki operasi dasar berupa push, pop, dan peek yang sangat penting dalam pengelolaan data. Selain itu, stack juga dapat diterapkan dalam berbagai permasalahan, seperti mencari nilai maksimum tanpa mengubah isi stack serta mengevaluasi ekspresi postfix secara efisien. Dengan demikian, pemahaman terhadap konsep dan implementasi stack sangat membantu dalam meningkatkan kemampuan pemrograman dan penyelesaian masalah secara sistematis.